**Independent University, Bangladesh**

CSE-313 Quiz 01 Exam, Autumn-2026, Sec: 01 · Total Marks: 15 · Time: 30min

Your real quiz. The answers follow the faculty solution sheet.

## Question paper

**1.** Consider the context-free grammar below. Is the grammar ambiguous or not? Find out a string for which it is ambiguous and convert to unambiguous grammar. **[7]**

```
L → L ; L | L & L | id
```

**2.** Convert the following Non Deterministic Grammar (NDG) to Deterministic Grammar (DG): **[4]**

```
S → a b c A | a b c B | a b | a b d
A → e A | e g f
B → g f
```

**3.** Convert the following Left Recursive Grammar (LRG) to Right Recursive Grammar (RRG): **[4]**

```
A → A a | B c | d
B → A e | C g h | g f
C → C a b | A c e | c d
```

## Answers (exam layout, with comments)

### Answer 1 [7]

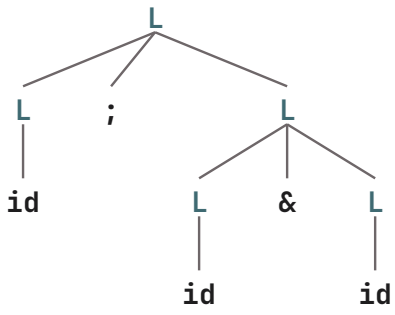
WRITE THIS

Yes, the grammar is ambiguous. String: `id ; id & id`

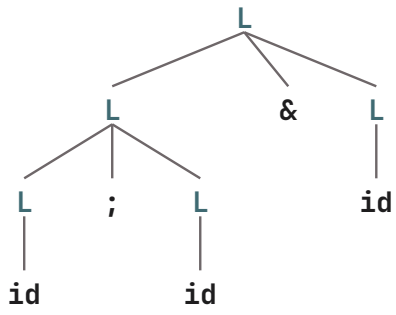
**LMD 1**: L ⇒ L ; L ⇒ id ; L ⇒ id ; L & L ⇒ id ; id & L ⇒ id ; id & id

**LMD 2**: L ⇒ L & L ⇒ L ; L & L ⇒ id ; L & L ⇒ id ; id & L ⇒ id ; id & id

Tree 1




Tree 2




Since for the string `id ; id & id` there exists more than one leftmost derivation parse tree, the grammar is ambiguous.

**Conversion to unambiguous grammar: [Assuming & has higher precedence than ; and both operators are left associative]**

```
L → L ; M | M
M → M & T | T
T → id
```

**Alt ans** [Assuming ; has higher precedence than & and both are left associative]:

```
L → L & M | M
M → M ; T | T
T → id
```

> **Lose marks by:** no string; two trees that are really the same; no assumption in brackets.

### Answer 2 [4]

WRITE THIS

**Round 1:** in S, `a b c A`, `a b c B`, `a b`, `a b d` all start with `a b`. Take it out once:

```
S → a b S'
S' → c A | c B | ε | d
```

**Round 2:** in S', `c A`, `c B` all start with `c`. Take it out once:

```
S' → c S'' | ε | d
S'' → A | B
```

**Round 3:** in A, `e A`, `e g f` all start with `e`. Take it out once:

```
A → e A'
A' → A | g f
```

**Final DG:**

```
S → a b S'
S' → c S'' | ε | d
S'' → A | B
A → e A'
A' → A | g f
B → g f
```

> **Lose marks by:** missing the second round on S'; forgetting to factor A.

## Answer 3 [4]

`WRITE THIS`

A → A a | B c | d: α = `a`, β = `B c`, `d`.

```
A → B c A' | d A'
A' → a A' | ε
```

B has an alternative that starts with A (already fixed), so hidden left recursion can come through it. Substitute A's current rules:

```
B → B c A' e | d A' e | C g h | g f
```

B → B c A' e | d A' e | C g h | g f: α = `c A' e`, β = `d A' e`, `C g h`, `g f`.

```
B → d A' e B' | C g h B' | g f B'
B' → c A' e B' | ε
```

C has an alternative that starts with A (already fixed), so hidden left recursion can come through it. Substitute A's current rules:

```
C → C a b | B c A' c e | d A' c e | c d
```

C has an alternative that starts with B (already fixed), so hidden left recursion can come through it. Substitute B's current rules:

```
C → C a b | d A' e B' c A' c e | C g h B' c A' c e | g f B' c A' c e | d A' c e | c d
```

C → C a b | d A' e B' c A' c e | C g h B' c A' c e | g f B' c A' c e | d A' c e | c d: α = `a b`, `g h B' c A' c e`, β = `d A' e B' c A' c e`, `g f B' c A' c e`, `d A' c e`, `c d`.

```
C → d A' e B' c A' c e C' | g f B' c A' c e C' | d A' c e C' | c d C'
C' → a b C' | g h B' c A' c e C' | ε
```

**Final RRG:**

```
A → B c A' | d A'
A' → a A' | ε
B → d A' e B' | C g h B' | g f B'
B' → c A' e B' | ε
C → d A' e B' c A' c e C' | g f B' c A' c e C' | d A' c e C' | c d C'
C' → a b C' | g h B' c A' c e C' | ε
```

Rule: A → Aα | β ⇒ A → βA', A' → αA' | ε. "β goes first, α goes to the new guy, new guy ends with ε."

> **Lose marks by:** missing the hidden (indirect) left recursion: B → A e and C → A c e must be substituted first.